Sri Lanka Institute of Information Technology

SE2020 – Web and Mobile Technologies
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 09

Student ID	IT24102099
Student Name	Mathuppriya Naguleswaran
Campus/ Center name	SLIIT – Northen UNI
Specialization	Software Engineering
Batch No	1

MongoDB

MongoDB

- It is a NoSQL document database.
- works well with JSON-like data in Node.js.

Mongoose

- It helps define schemas, validations, and relationships more easily.

Why create separate models?

- Because Student, MenuItem, and Order are different entities in the system.

Why use references in Order?

- Because one order belongs to a student and contains menu items from another collection.

populate()

- To fetch full related document details instead of only IDs.

Why use aggregation?

- To calculate summaries such as total spent, top selling items, and daily order count efficiently in the database.

Why use pagination?

- To avoid returning too many orders at once and improve performance.

Why use regex in search?

- To support partial and case-insensitive searching.

Step 1: Create the project folder.

Step 2: Open the folder in VS Code.

Step 3: Open terminal in VS Code.

Step 4: Initialize Node project.

npm init -y

- This creates a package.json file.

```
PS C:\Users\vallu\Desktop\campus-food-api_It24102099> npm init -y
Wrote to C:\Users\vallu\Desktop\campus-food-api_It24102099\package.json:

{
  "name": "campus-food-api_it24102099",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

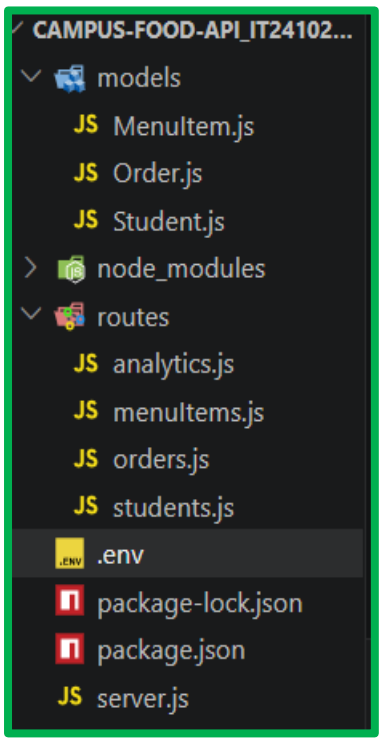
Step 5: Install required packages

npm install express mongoose cors dotenv

- express → to create the API server
- mongoose → to connect Node.js with MongoDB
- cors → to allow cross-origin requests
- dotenv → to load values from .env

```
PS C:\Users\vallu\Desktop\campus-food-api_It24102099> npm install express mongoose cors dotenv

added 85 packages, and audited 86 packages in 14s

25 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\vallu\Desktop\campus-food-api_It24102099>
```

Step 6: Create file and folder structure



Step 7: Setup MongoDB connection

For the data base im using local MongoDB. So I don't have any password for that. If I use MongoDB Atlas **must** have database username , database password, cluster connection string.

In .env file type this,

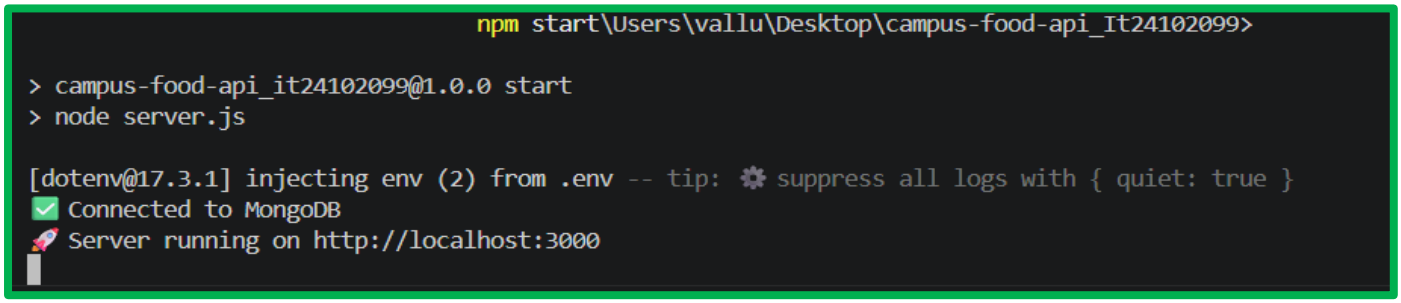
PORT=3000

MONGO_URI=mongodb://127.0.0.1:27017/campus_food

Or

MONGO_URI=mongodb://localhost:27017/campus_food

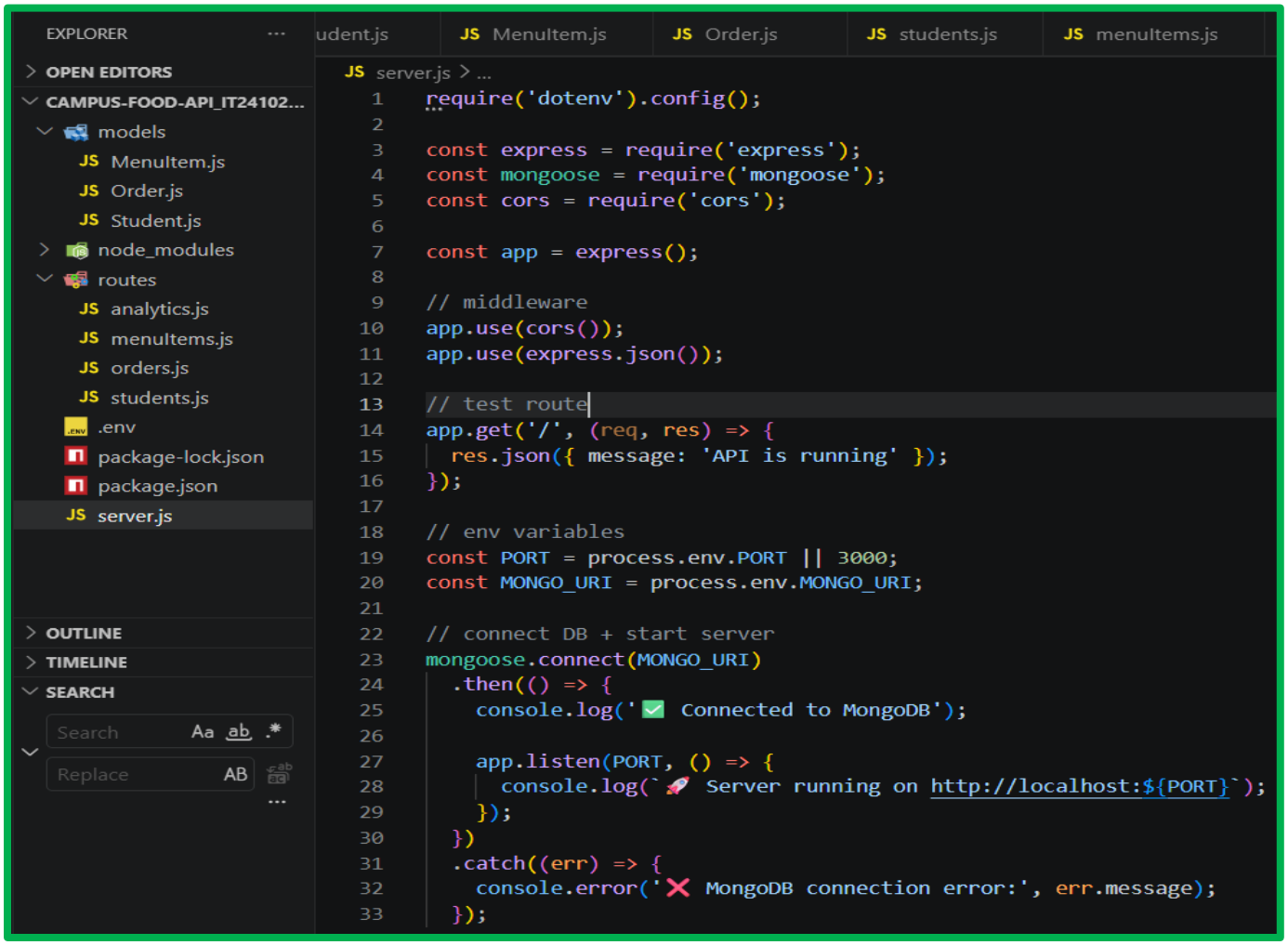
- localhost = your own computer
- 27017 = default MongoDB port
- campus_food = database name, MongoDB will create it when data is inserted



Step 8: Write server.js

Run the server easily with one command.

- npm start or node server.js



- **require('dotenv').config();** : Loads values from .env.
- **express** : Creates the server.
- **Mongoose** : Connects to MongoDB.
- **Cors :** Allows requests from other apps or tools.
- **app.use(express.json()) :** server read JSON body data from Postman.
- **app.get('/')** : Creates a simple route to test whether server works.
- **app.use('/students', studentRoutes)** : Mounts student routes.
- **mongoose.connect(MONGO_URI)** : Connects to database.

Step 9: Add start script in package.json

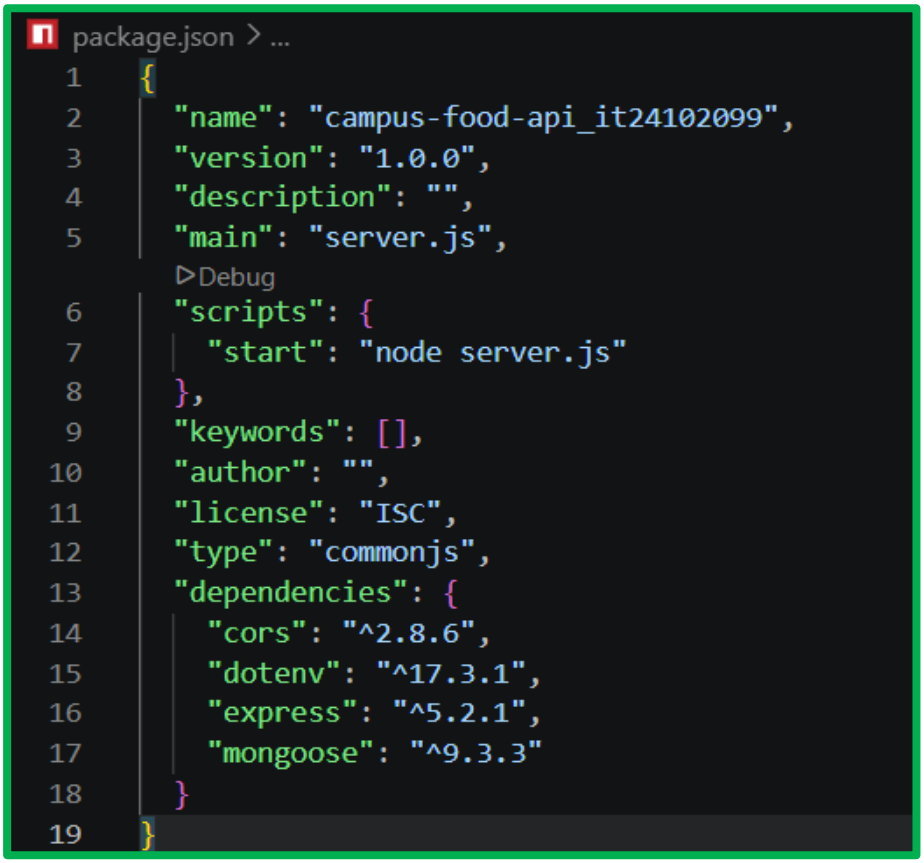
When open the package.json its have this version of code

But here I want to change and add extra things related to my project

My project have server.js NOT index.js

And add **"start": "node server.js"**

- This tells npm what to run when you type: npm start



If you want auto restart during development, install nodemon:

```
npm install --save-dev nodemon
```

Then use:

```
"scripts": {
  "start": "node server.js",
  "dev": "nodemon server.js"
}
```

So we can start server easily with one command.

Step 10: Run the server

```
                        npm start\Users\vallu\Desktop\campus-food-api_It24102099>

> campus-food-api_it24102099@1.0.0 start
> node server.js

[dotenv@17.3.1] injecting env (2) from .env -- tip: ⚙ suppress all logs with { quiet: true }
✅ Connected to MongoDB
🚀 Server running on http://localhost:3000
```

Step 11: Create the Student model

```
models > JS Student.js > ...
1   const mongoose = require('mongoose');
2
3   const studentSchema = new mongoose.Schema(
4     {
5       name: {
6         type: String,
7         required: true,
8         trim: true
9       },
10      email: {
11        type: String,
12        required: true,
13        unique: true,
14        trim: true
15      },
16      faculty: {
17        type: String,
18        trim: true
19      },
20      year: {
21        type: Number,
22        min: 1,
23        max: 4
24      }
25    },
26    {
27      timestamps: true
28    }
29  );
30
31  const Student = mongoose.model('Student', studentSchema);
32
33  module.exports = Student;
34
```

- required: true => Field cannot be empty.
- unique: true =>Two students cannot have same email.
- trim: true =>Removes extra spaces.
- min: 1, max: 4 =>Year must be between 1 and 4.
- timestamps: true =>Mongoose automatically adds createdAt and updatedAt.

Step 12: Create the MenuItem model

models > JS MenuItem.js > ...

```
const mongoose = require('mongoose');

const menuItemSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true
    },
    price: {
      type: Number,
      required: true,
      min: 0
    },
    category: {
      type: String,
      trim: true
    },
    isAvailable: {
      type: Boolean,
      default: true
    }
  },
  {
    timestamps: true
  }
);

const MenuItem = mongoose.model('MenuItem', menuItemSchema);

module.exports = MenuItem;
```

Step 13: Create the Order model

This is the first time you are using **references**.

- student stores the ObjectId of a Student
- menuItem stores the ObjectId of a MenuItem

So Order connects all collections together.

models > JS Order.js > ...

```
const mongoose = require('mongoose');

const orderItemSchema = new mongoose.Schema(
  {
    menuItem: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'MenuItem',
      required: true
    },
    quantity: {
      type: Number,
      required: true,
      min: 1
    }
  },
  {
    _id: false
  }
);

const orderSchema = new mongoose.Schema({
  student: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Student',
    required: true
  },
  items: {
    type: [orderItemSchema],
    validate: [(val) => val.length > 0, 'At least one item is required']
  },
  totalPrice: {
    type: Number,
    required: true,
    min: 0
  },
  status: {
    type: String,
    enum: ['PLACED', 'PREPARING', 'DELIVERED', 'CANCELLED'],
    default: 'PLACED'
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

const Order = mongoose.model('Order', orderSchema);

module.exports = Order;

```

Step 14: Create student routes

routes > JS students.js > ...

```
const express = require('express');
const Student = require('../models/Student');

const router = express.Router();

// POST /students
router.post('/', async (req, res) => {
  try {
    const student = new Student(req.body);
    const saved = await student.save();
    res.status(201).json(saved);
  } catch (err) {
    console.error('Error creating student:', err.message);
    res.status(400).json({ error: err.message });
  }
});

// GET /students/:id
router.get('/:id', async (req, res) => {
  try {
    const student = await Student.findById(req.params.id);

    if (!student) {
      return res.status(404).json({ error: 'Student not found' });
    }

    res.json(student);
  } catch (err) {
    console.error('Error fetching student:', err.message);
    res.status(400).json({ error: 'Invalid student ID' });
  }
});

module.exports = router;
```

Step 15: Create menu item routes

The search should support partial case-insensitive name match.
{ $regex: name, $options: 'i' }
This means:

- search partially
- ignore uppercase/lowercase

So egg can match Egg Fried Rice.

routes > JS menuItems.js > ...

```js
const express = require('express');
const MenuItem = require('../models/MenuItem');

const router = express.Router();

// POST /menu-items
router.post('/', async (req, res) => {
  try {
    const menuItem = new MenuItem(req.body);
    const saved = await menuItem.save();
    res.status(201).json(saved);
  } catch (err) {
    console.error('Error creating menu item:', err.message);
    res.status(400).json({ error: err.message });
  }
});

// GET /menu-items
router.get('/', async (req, res) => {
  try {
    const items = await MenuItem.find().sort({ createdAt: -1 });
    res.json(items);
  } catch (err) {
    console.error('Error fetching menu items:', err.message);
    res.status(500).json({ error: 'Server error' });
  }
});
```

```js
// GET /menu-items/search?name=...&category=...
router.get('/search', async (req, res) => {
  try {
    const { name, category } = req.query;

    const filter = {};

    if (name) {
      filter.name = { $regex: name, $options: 'i' };
    }

    if (category) {
      filter.category = category;
    }

    const items = await MenuItem.find(filter).sort({ name: 1 });
    res.json(items);
  } catch (err) {
    console.error('Error searching menu items:', err.message);
    res.status(500).json({ error: 'Server error' });
  }
});

module.exports = router;
```

Step 16: Create order routes

❖ populate()

- Without populate, you only get IDs.

  With populate, you get full student and menu item details.

  Example:

  Instead of only this:

  "student": "abc123"

  you get:

  "student": {

  "_id": "abc123",

  "name": "Alice"

  }

routes > JS orders.js > calculateTotalPrice

```
const express = require('express');
const Order = require('../models/Order');
const MenuItem = require('../models/MenuItem');
const Student = require('../models/Student');

const router = express.Router();

async function calculateTotalPrice(items) {
  const menuItemIds = items.map(item => item.menuItem);

  const menuItems = await MenuItem.find({ _id: { $in: menuItemIds } });

  const priceMap = {};
  menuItems.forEach(item => {
    priceMap[item._id.toString()] = item.price;
  });

  let total = 0;

  for (const item of items) {
    const price = priceMap[item.menuItem.toString()];
    if (price === undefined) {
      throw new Error(`Invalid menuItem ID: ${item.menuItem}`);
    }
    total += price * item.quantity;
  }

  return total;
}

```

```
// POST /orders
router.post('/', async (req, res) => {
  try {
    const { student, items } = req.body;

    if (!student) {
      return res.status(400).json({ error: 'Student is required' });
    }

    const studentExists = await Student.findById(student);
    if (!studentExists) {
      return res.status(400).json({ error: 'Student not found' });
    }

    if (!Array.isArray(items) || items.length === 0) {
      return res.status(400).json({ error: 'Items must be a non-empty array' });
    }

    const totalPrice = await calculateTotalPrice(items);

    const order = new Order({
      student,
      items,
      totalPrice,
      status: 'PLACED'
    });

    const saved = await order.save();

```

```

      const populatedOrder = await Order.findById(saved._id)
        .populate('student')
        .populate('items.menuItem');

      res.status(201).json(populatedOrder);
    } catch (err) {
      console.error('Error placing order:', err.message);
      res.status(400).json({ error: err.message });
    }
  });

  // GET /orders?page=1&limit=2
  router.get('/', async (req, res) => {
    try {
      const page = Number(req.query.page) || 1;
      const limit = Number(req.query.limit) || 10;
      const skip = (page - 1) * limit;

      const orders = await Order.find()
        .sort({ createdAt: -1 })
        .skip(skip)
        .limit(limit)
        .populate('student')
        .populate('items.menuItem');

      const totalOrders = await Order.countDocuments();
      const totalPages = Math.ceil(totalOrders / limit);

```

```
87
88        res.json({
89          page,
90          limit,
91          totalOrders,
92          totalPages,
93          orders
94        });
95      } catch (err) {
96        console.error('Error fetching orders:', err.message);
97        res.status(500).json({ error: 'Server error' });
98      }
99    });
100
101   // GET /orders/:id
102   router.get('/:id', async (req, res) => {
103     try {
104       const order = await Order.findById(req.params.id)
105         .populate('student')
106         .populate('items.menuItem');
107
108       if (!order) {
109         return res.status(404).json({ error: 'Order not found' });
110       }
111
112       res.json(order);
113     } catch (err) {
114       console.error('Error fetching order:', err.message);
115       res.status(400).json({ error: 'Invalid order ID' });
116     }
117   });
```

```
118
119   // PATCH /orders/:id/status
120   router.patch('/:id/status', async (req, res) => {
121     try {
122       const { status } = req.body;
123       const allowedStatuses = ['PLACED', 'PREPARING', 'DELIVERED', 'CANCELLED'];
124
125       if (!allowedStatuses.includes(status)) {
126         return res.status(400).json({ error: 'Invalid status value' });
127       }
128
129       const order = await Order.findByIdAndUpdate(
130         req.params.id,
131         { status },
132         { new: true, runValidators: true }
133       )
134         .populate('student')
135         .populate('items.menuItem');
136
137       if (!order) {
138         return res.status(404).json({ error: 'Order not found' });
139       }
140
141       res.json(order);
142     } catch (err) {
143       console.error('Error updating order status:', err.message);
144       res.status(400).json({ error: err.message });
145     }
146   });
147
```

```
148  // DELETE /orders/:id
149  router.delete('/:id', async (req, res) => {
150    try {
151      const deleted = await Order.findByIdAndDelete(req.params.id);
152
153      if (!deleted) {
154        return res.status(404).json({ error: 'Order not found' });
155      }
156
157      res.json({ message: 'Order deleted successfully' });
158    } catch (err) {
159      console.error('Error deleting order:', err.message);
160      res.status(400).json({ error: 'Invalid order ID' });
161    }
162  });
163
164  module.exports = router;
```

Step 17: Create analytics routes

routes > JS analytics.js > router.get('/total-spent/:studentId') callback

```
1   const express = require('express');
2   const mongoose = require('mongoose');
3   const Order = require('../models/Order');
4   const MenuItem = require('../models/MenuItem');
5
6   const router = express.Router();
7
8   // GET /analytics/total-spent/:studentId
9   router.get('/total-spent/:studentId', async (req, res) => {
10    try {
11      const { studentId } = req.params;
12
13      if (!mongoose.Types.ObjectId.isValid(studentId)) {
14        return res.status(400).json({ error: 'Invalid student ID' });
15      }
16
17      const result = await Order.aggregate([
18        {
19          $match: {
20            student: new mongoose.Types.ObjectId(studentId)
21          }
22        },
23        {
24          $group: {
25            _id: '$student',
26            totalSpent: { $sum: '$totalPrice' }
27          }
28        }
29      ]);
30
```

```
30
31      if (result.length === 0) {
32        return res.json({ studentId, totalSpent: 0 });
33      }
34
35      res.json({
36        studentId,
37        totalSpent: result[0].totalSpent
38      });
39    } catch (err) {
40      console.error('Error calculating total spent:', err.message);
41      res.status(500).json({ error: 'Server error' });
42    }
43  });
44
```

```
45  // GET /analytics/top-menu-items?limit=5
46  router.get('/top-menu-items', async (req, res) => {
47    try {
48      const limit = Number(req.query.limit) || 5;
49
50      const result = await Order.aggregate([
51        { $unwind: '$items' },
52        {
53          $group: {
54            _id: '$items.menuItem',
55            totalQuantity: { $sum: '$items.quantity' }
56          }
57        },
58        { $sort: { totalQuantity: -1 } },
59        { $limit: limit }
60      ]);
61
62      const formatted = [];
63
64      for (const item of result) {
65        const menuItem = await MenuItem.findById(item._id);
66        formatted.push({
67          menuItem,
68          totalQuantity: item.totalQuantity
69        });
70      }
71
```

```
71
72      res.json(formatted);
73    } catch (err) {
74      console.error('Error fetching top menu items:', err.message);
75      res.status(500).json({ error: 'Server error' });
76    }
77  });
78
79  // GET /analytics/daily-orders
80  router.get('/daily-orders', async (req, res) => {
81    try {
82      const result = await Order.aggregate([
83        {
84          $group: {
85            _id: {
86              $dateToString: { format: '%Y-%m-%d', date: '$createdAt' }
87            },
88            orderCount: { $sum: 1 }
89          }
90        },
91        { $sort: { _id: 1 } }
92      ]);
93
94      const formatted = result.map(item => ({
95        date: item._id,
96        orderCount: item.orderCount
97      }));
98
99      res.json(formatted);
100   } catch (err) {
101     console.error('Error fetching daily orders:', err.message);
102     res.status(500).json({ error: 'Server error' });
103   }
104 });
105
106 module.exports = router;
```

requires these analytics endpoints:

- total amount spent by a student
- top selling menu items
- daily order counts

and it specifically says to use MongoDB aggregation pipeline.

$match : Filters documents.

$group : Groups records and calculates totals.

$sum : Adds values.

$unwind : Breaks an array into separate records so each item can be counted.

$dateToString : Converts date into YYYY-MM-DD format.

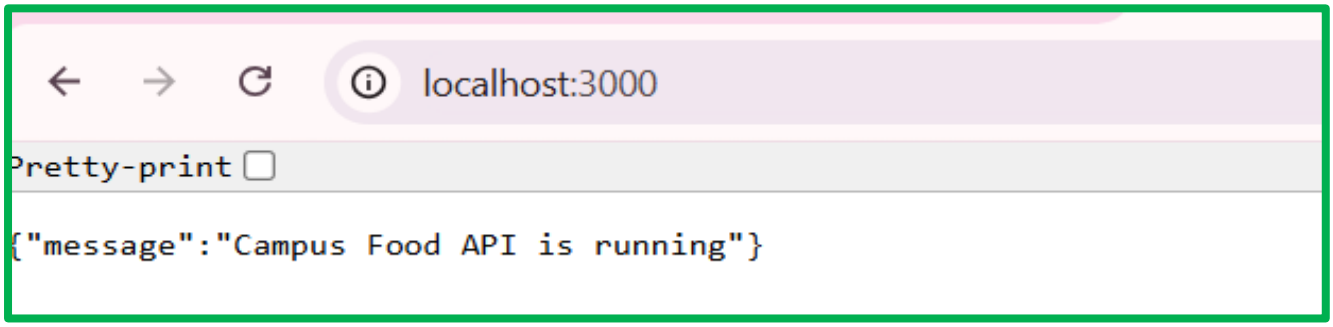
Step 18: Run server again

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\vallu\Desktop\campus-food-api_It24102099> npm start

> campus-food-api_it24102099@1.0.0 start
> node server.js

[dotenv@17.3.1] injecting env (2) from .env -- tip: 🔐 prevent building .env in docker: https://dotenvx.com/prebuild
✅ Connected to MongoDB
🚀 Server running on http://localhost:3000
```

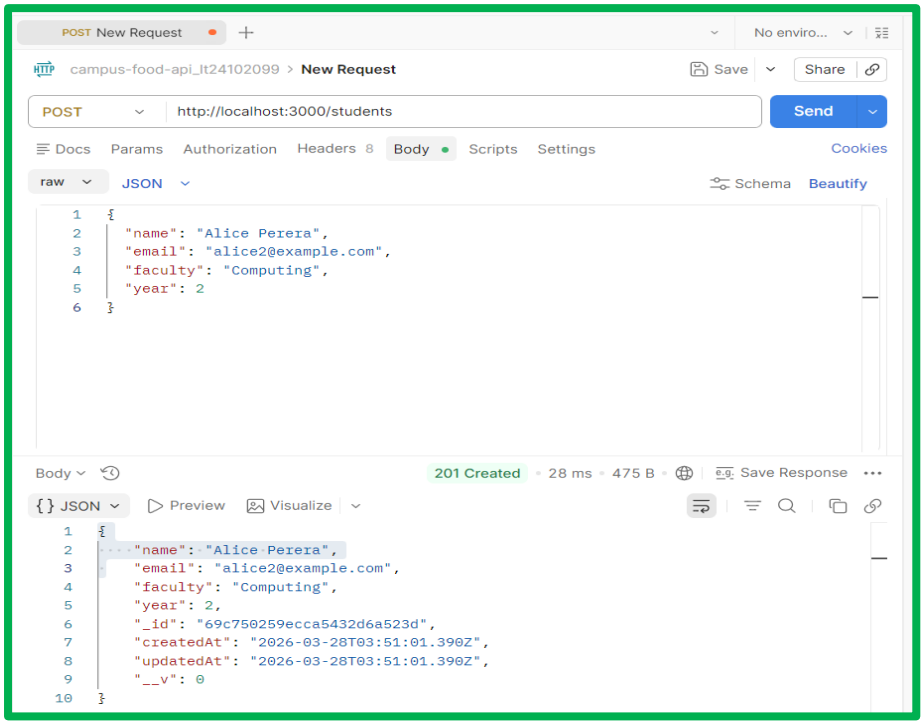
In Node.js:
You cannot declare the same variable twice.

Step 19: Test in Postman

1. Test root route

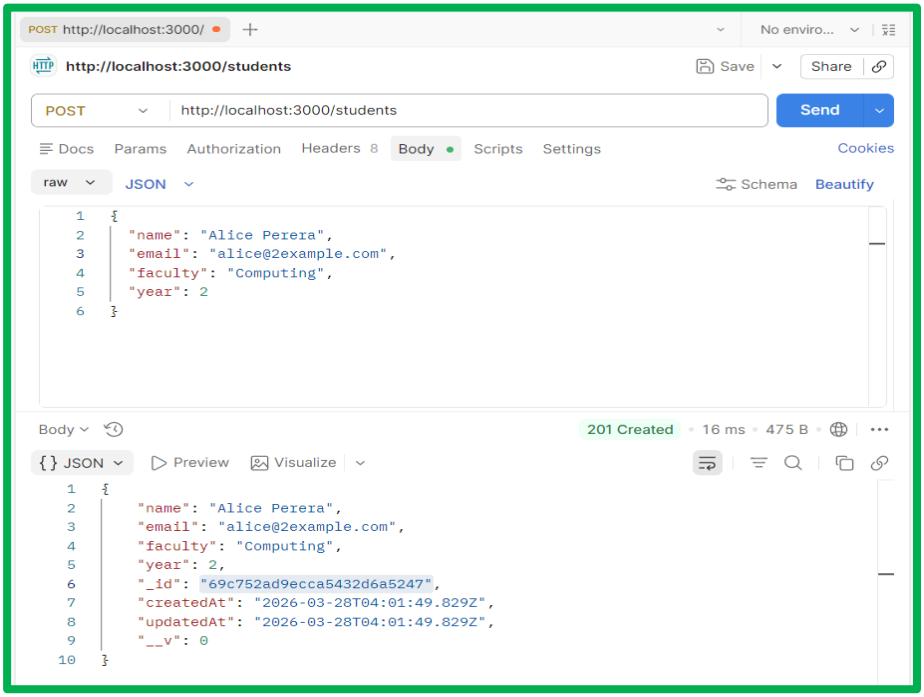


2.

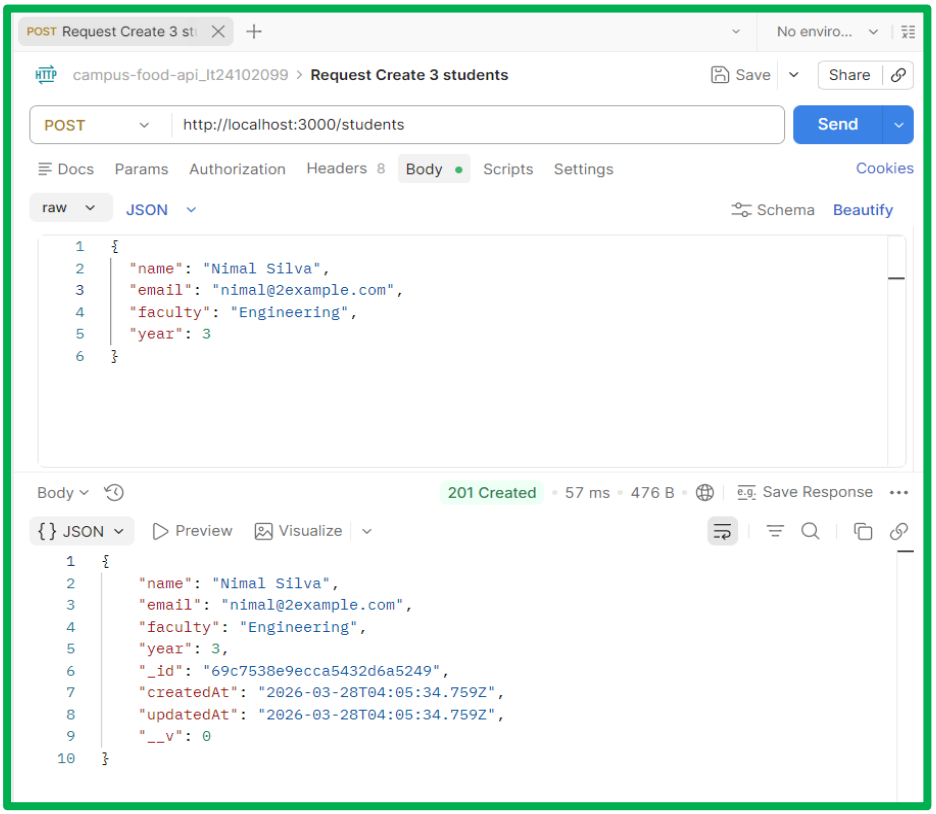


Add new collection -> Rename with a name -> Select request type (post) -> In body select raw -> Then write it. -> click send.

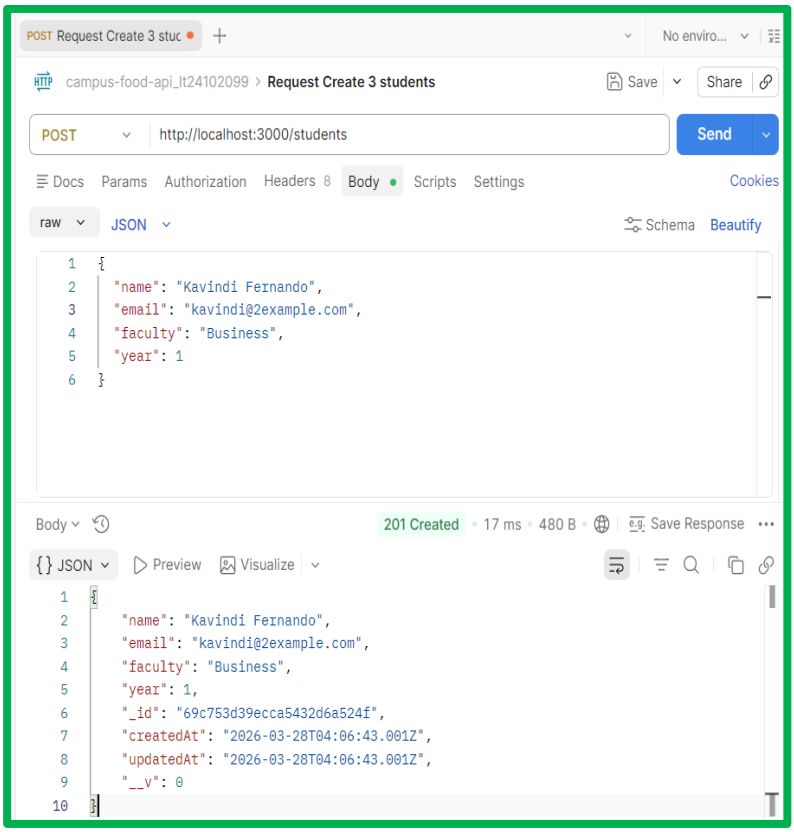
Student 1:



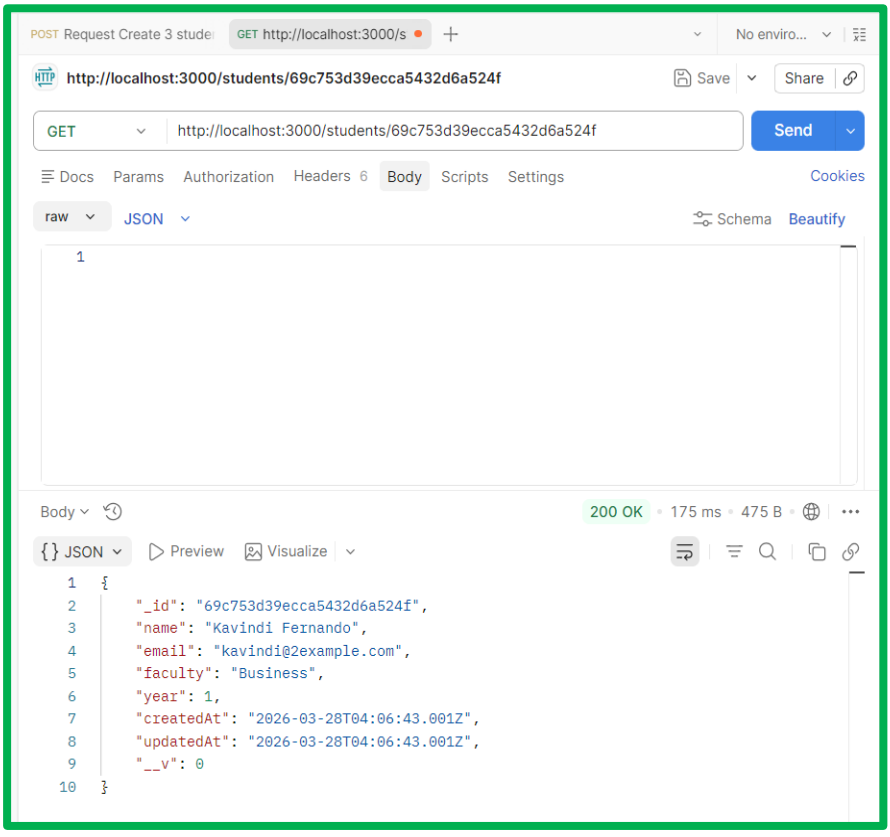
Student 2:



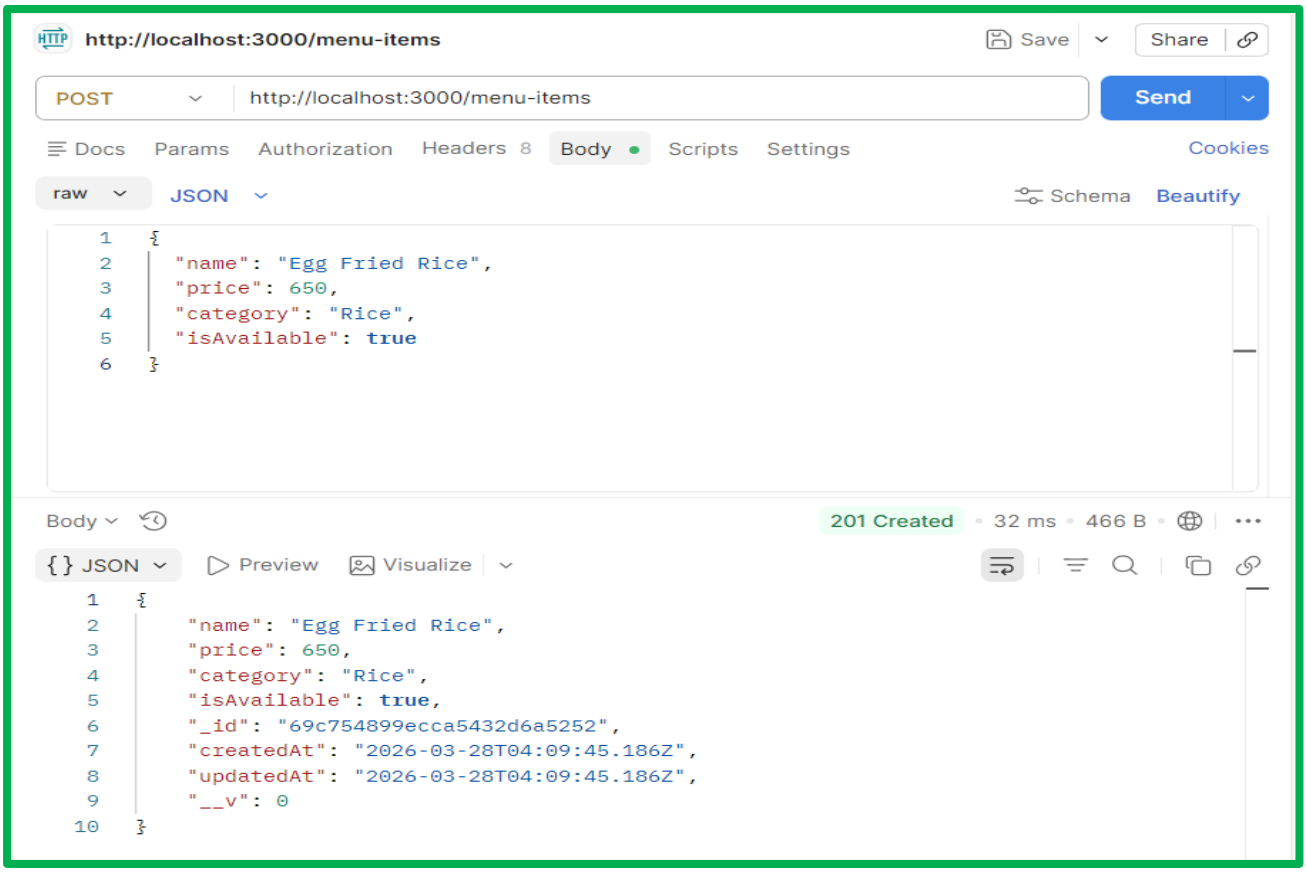
Student 3:

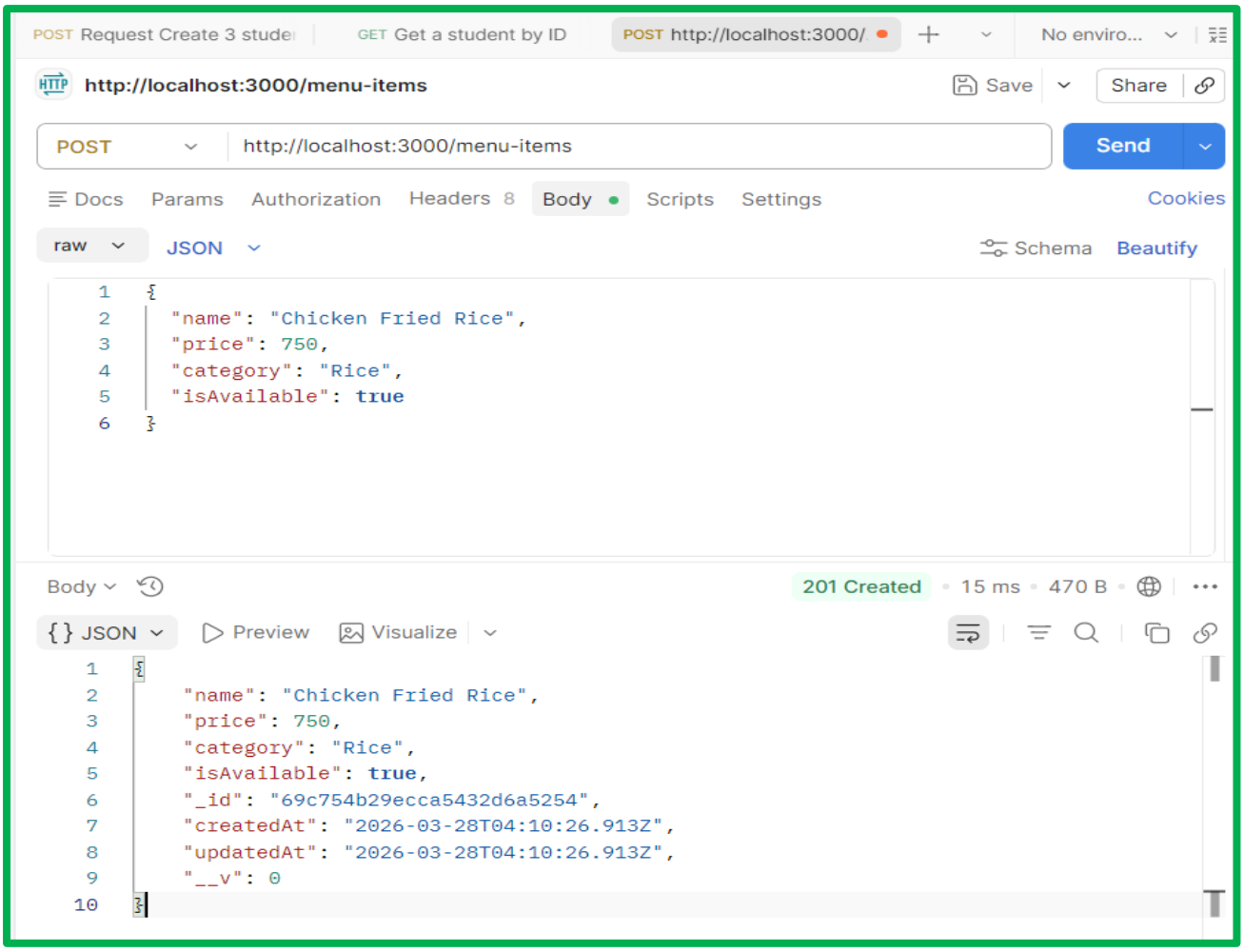


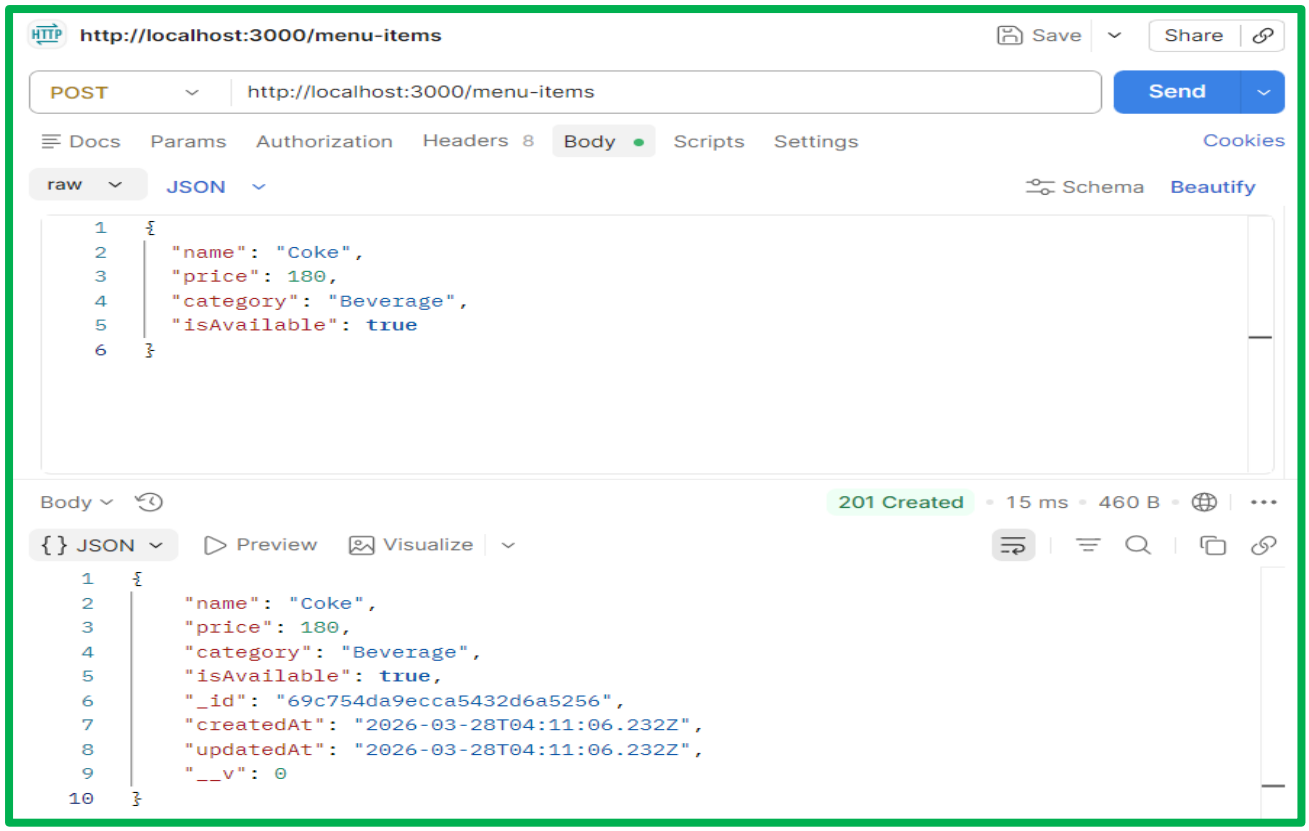
Get a student by ID

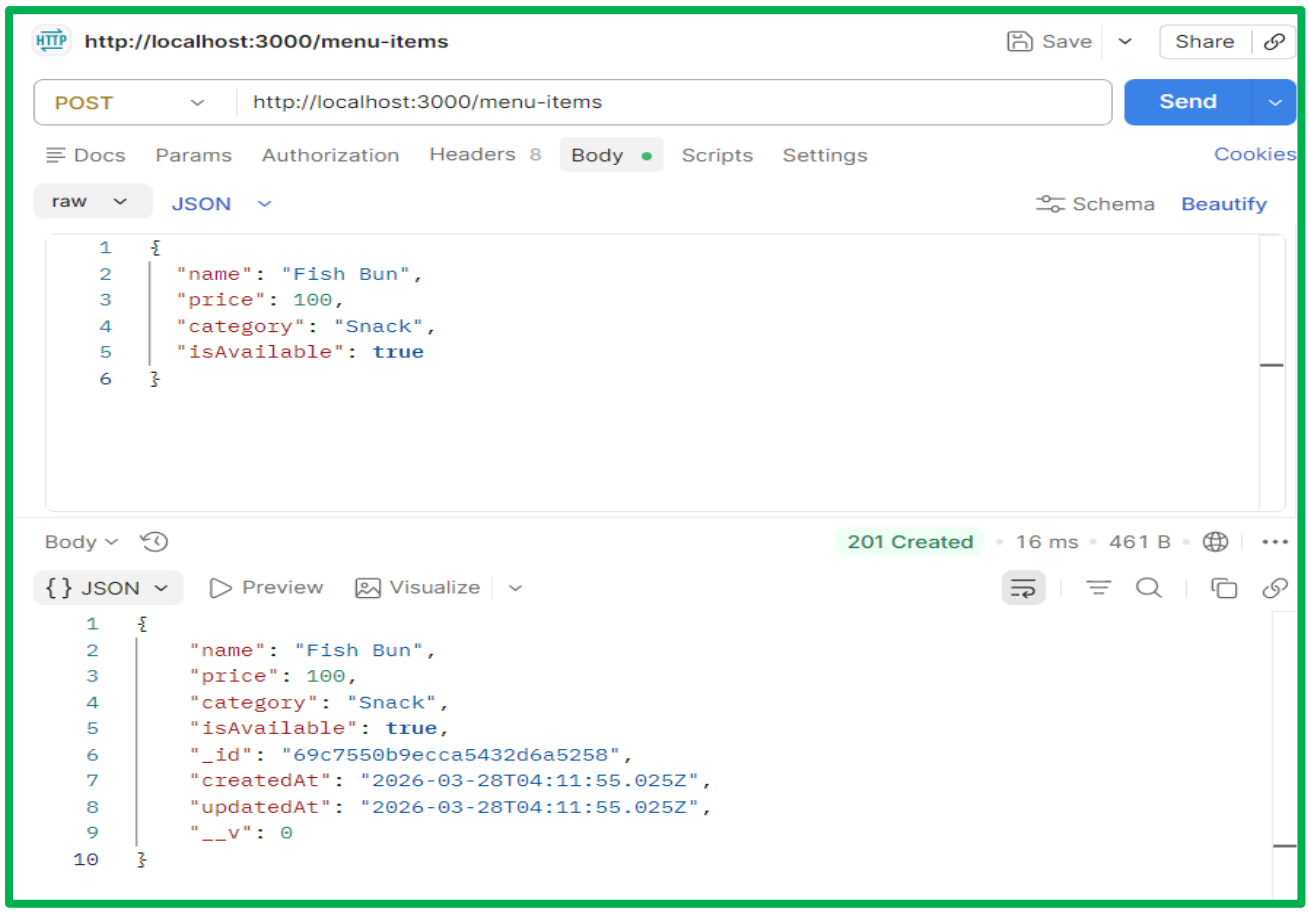


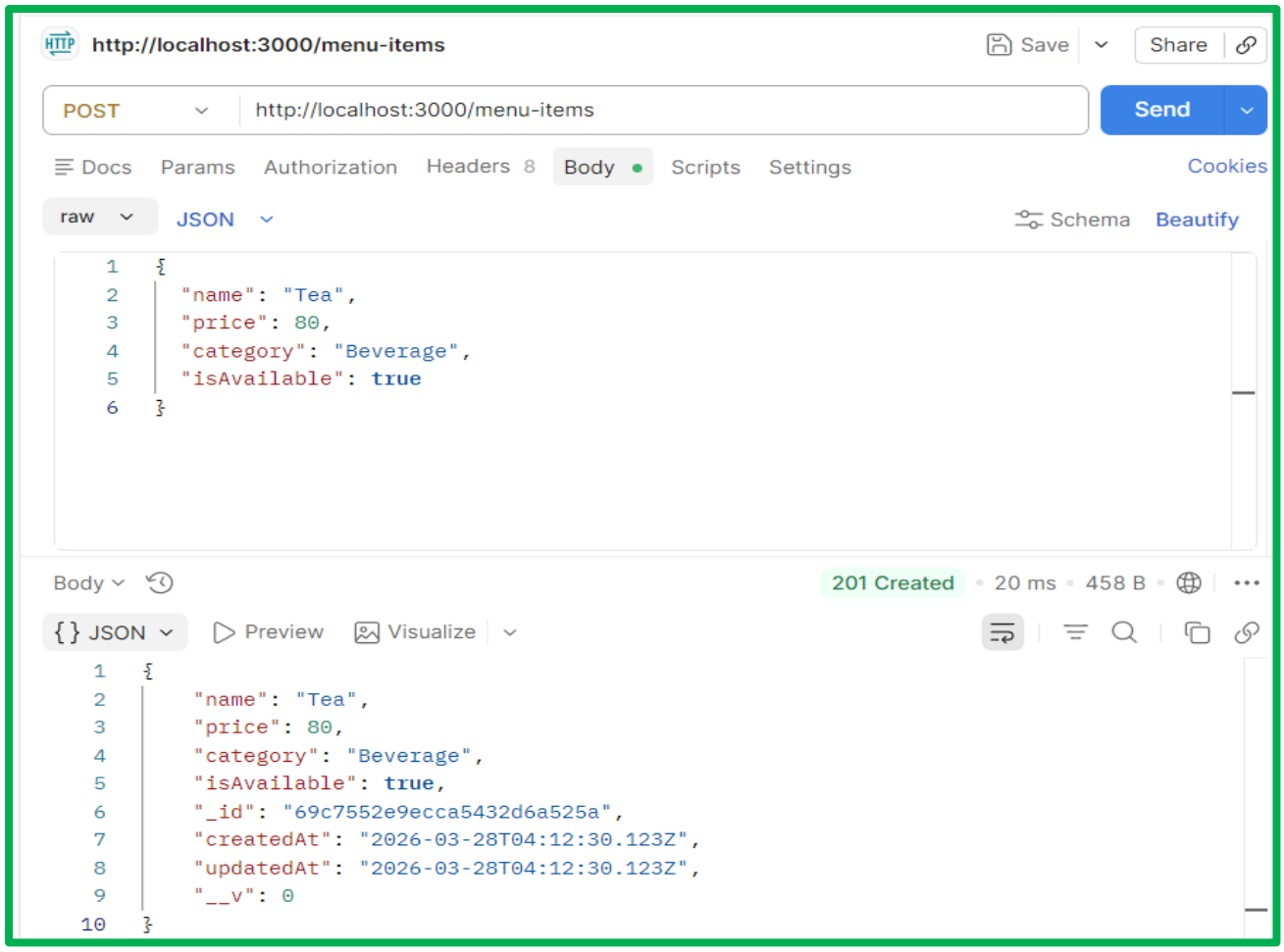
Add menu item



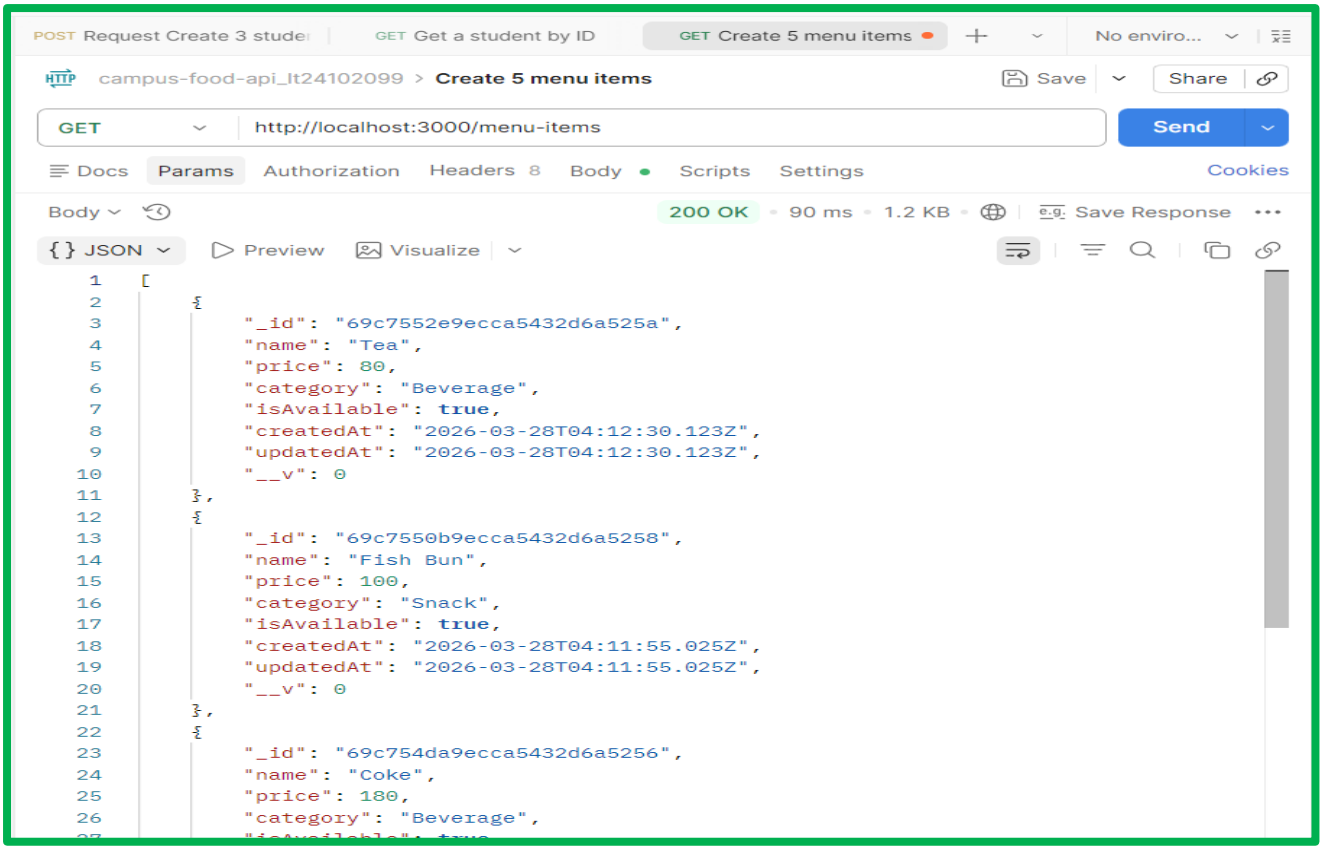



http://localhost:3000/menu-items
Save
Share
POST
http://localhost:3000/menu-items
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"name": "Coke",
"price": 180,
"category": "Beverage",
"isAvailable": true
}
Body
201 Created
15 ms
460 B
{} JSON
Preview
Visualize
{
"name": "Coke",
"price": 180,
"category": "Beverage",
"isAvailable": true,
"_id": "69c754da9ecca5432d6a5256",
"createdAt": "2026-03-28T04:11:06.232Z",
"updatedAt": "2026-03-28T04:11:06.232Z",
"__v": 0
}


http://localhost:3000/menu-items
Save
Share
POST
http://localhost:3000/menu-items
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"name": "Fish Bun",
"price": 100,
"category": "Snack",
"isAvailable": true
}
Body
201 Created
16 ms
461 B
{} JSON
Preview
Visualize
{
"name": "Fish Bun",
"price": 100,
"category": "Snack",
"isAvailable": true,
"_id": "69c7550b9ecca5432d6a5258",
"createdAt": "2026-03-28T04:11:55.025Z",
"updatedAt": "2026-03-28T04:11:55.025Z",
"__v": 0
}

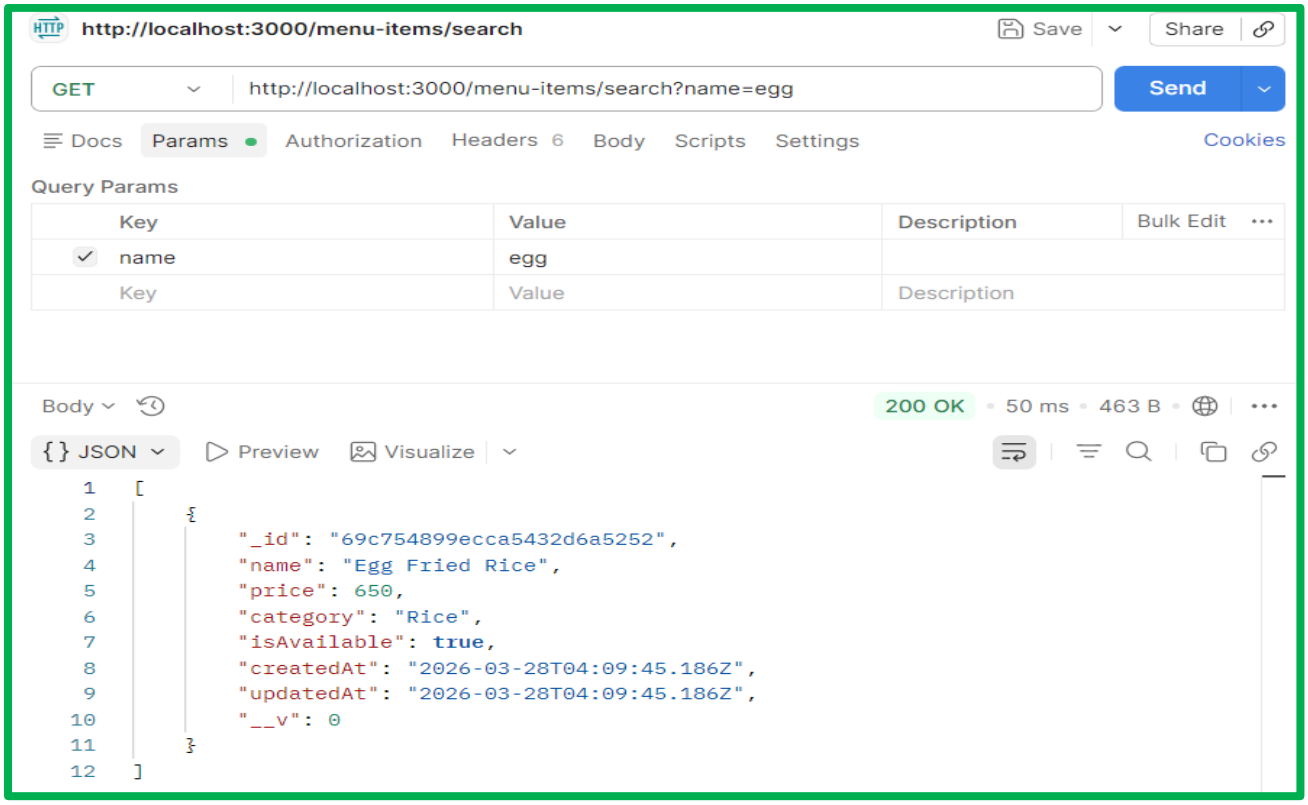


List menu items

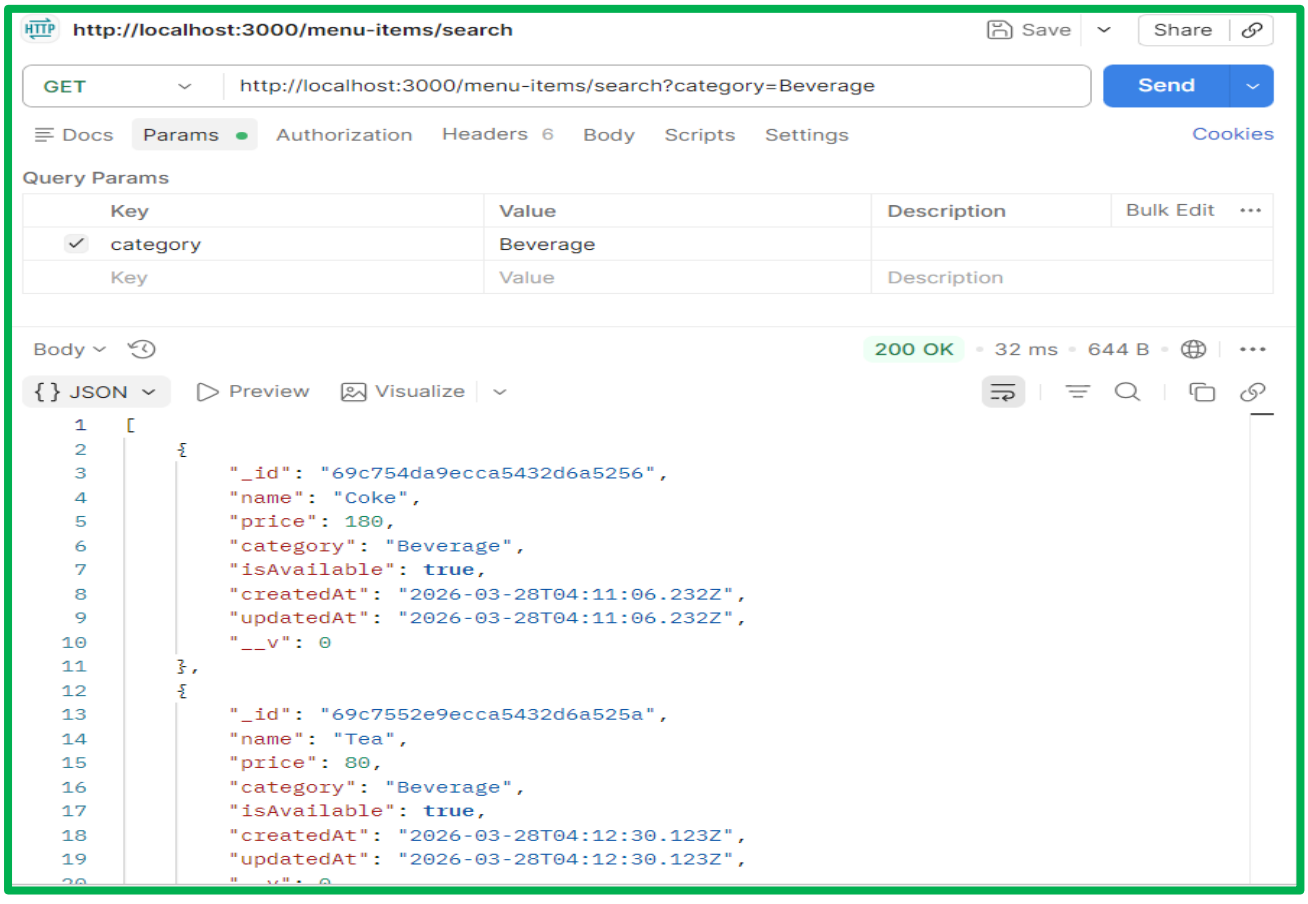


Search menu items

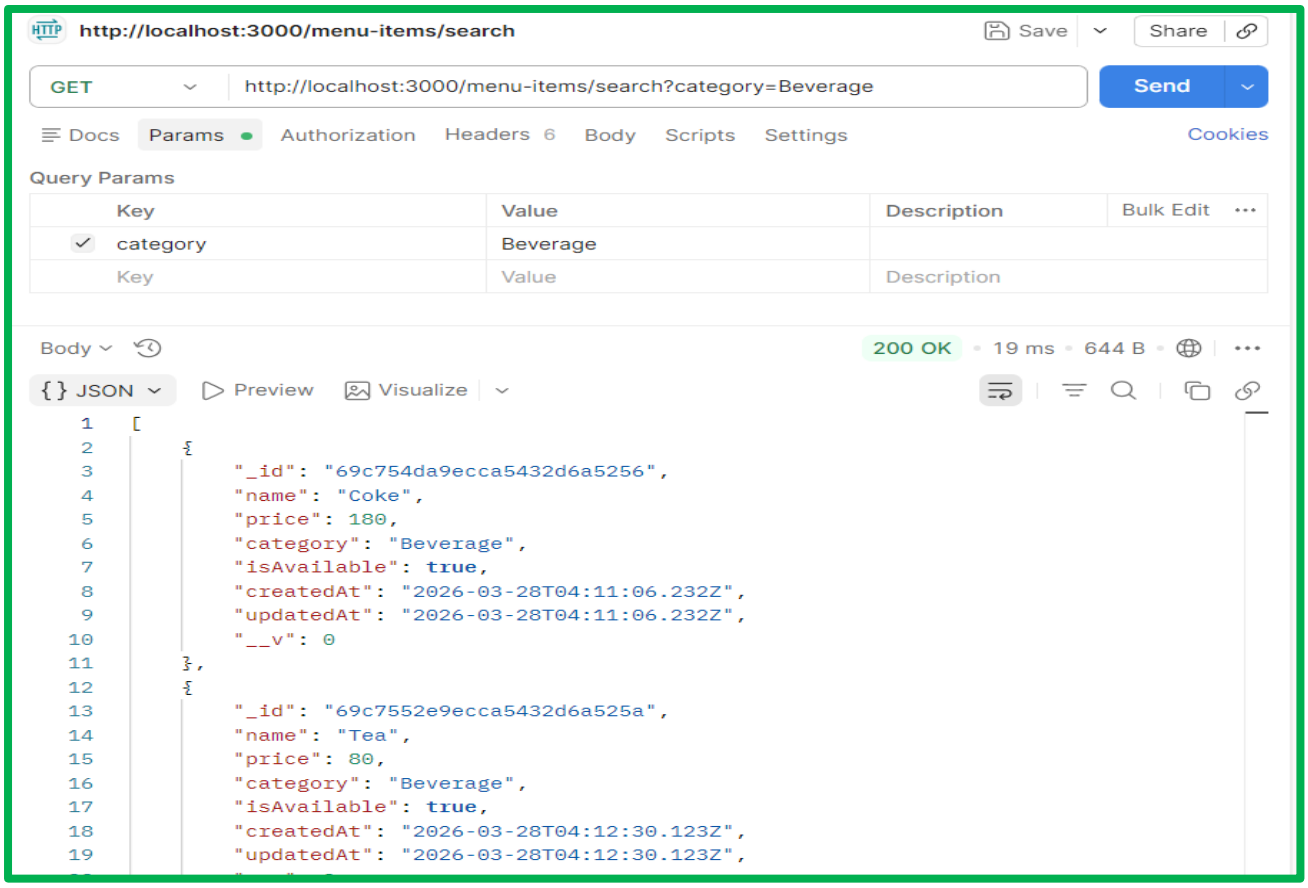
By name:



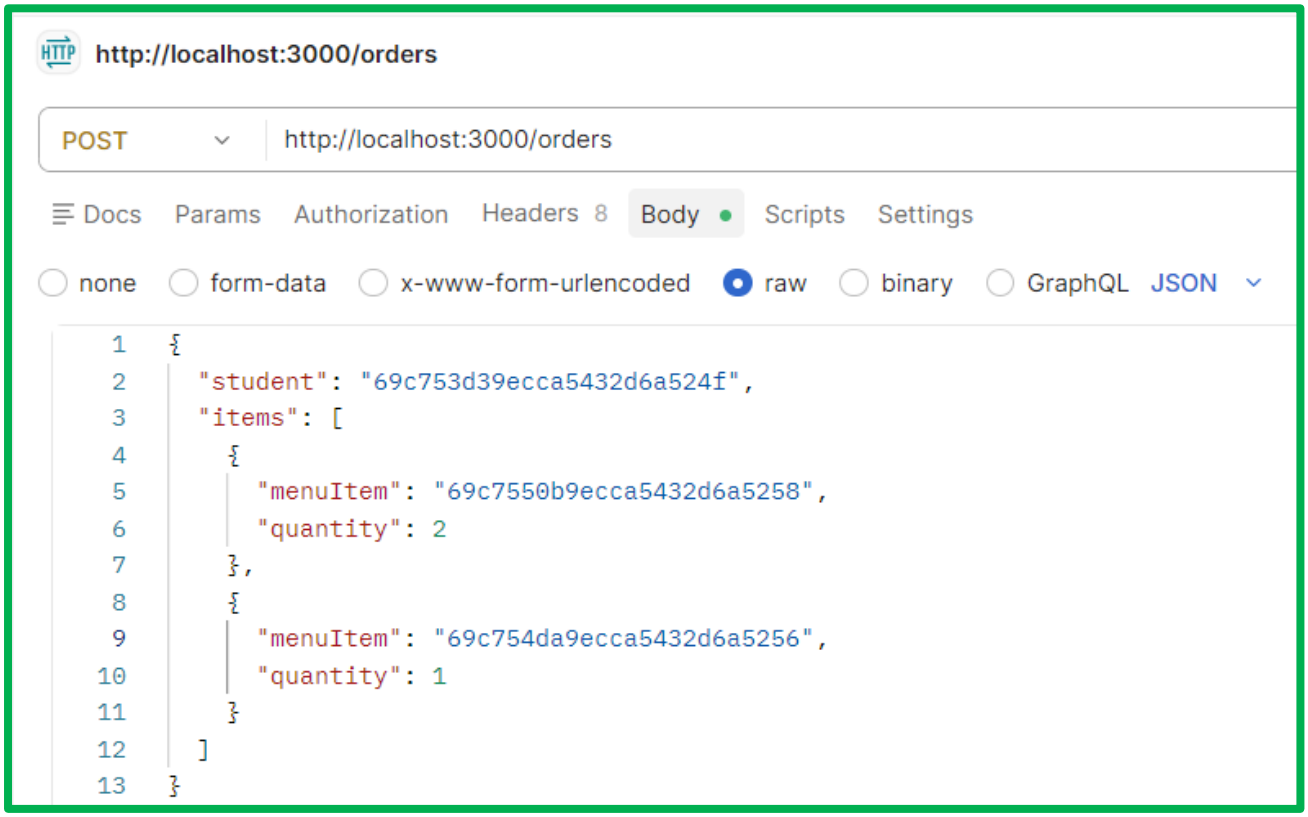
By category:

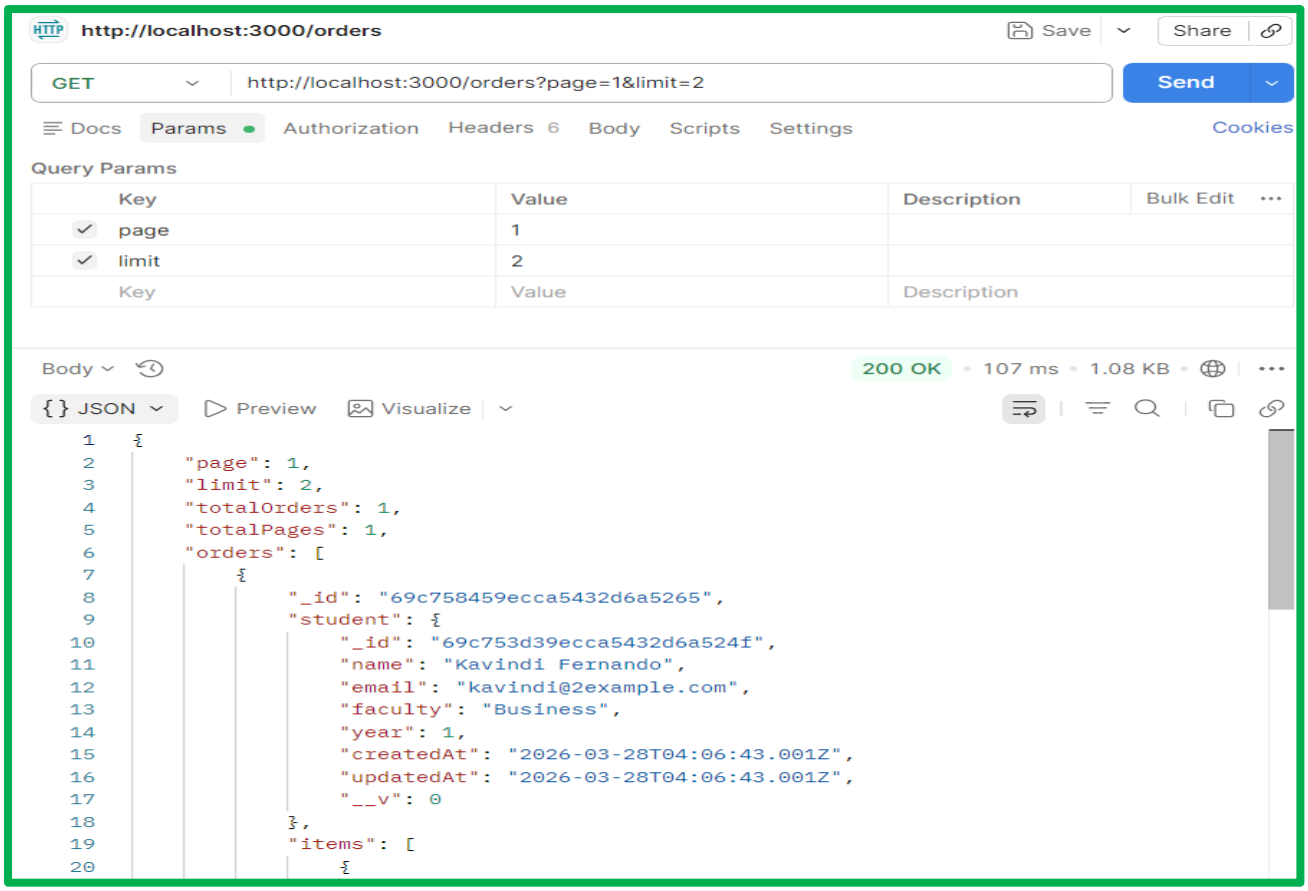


By both:

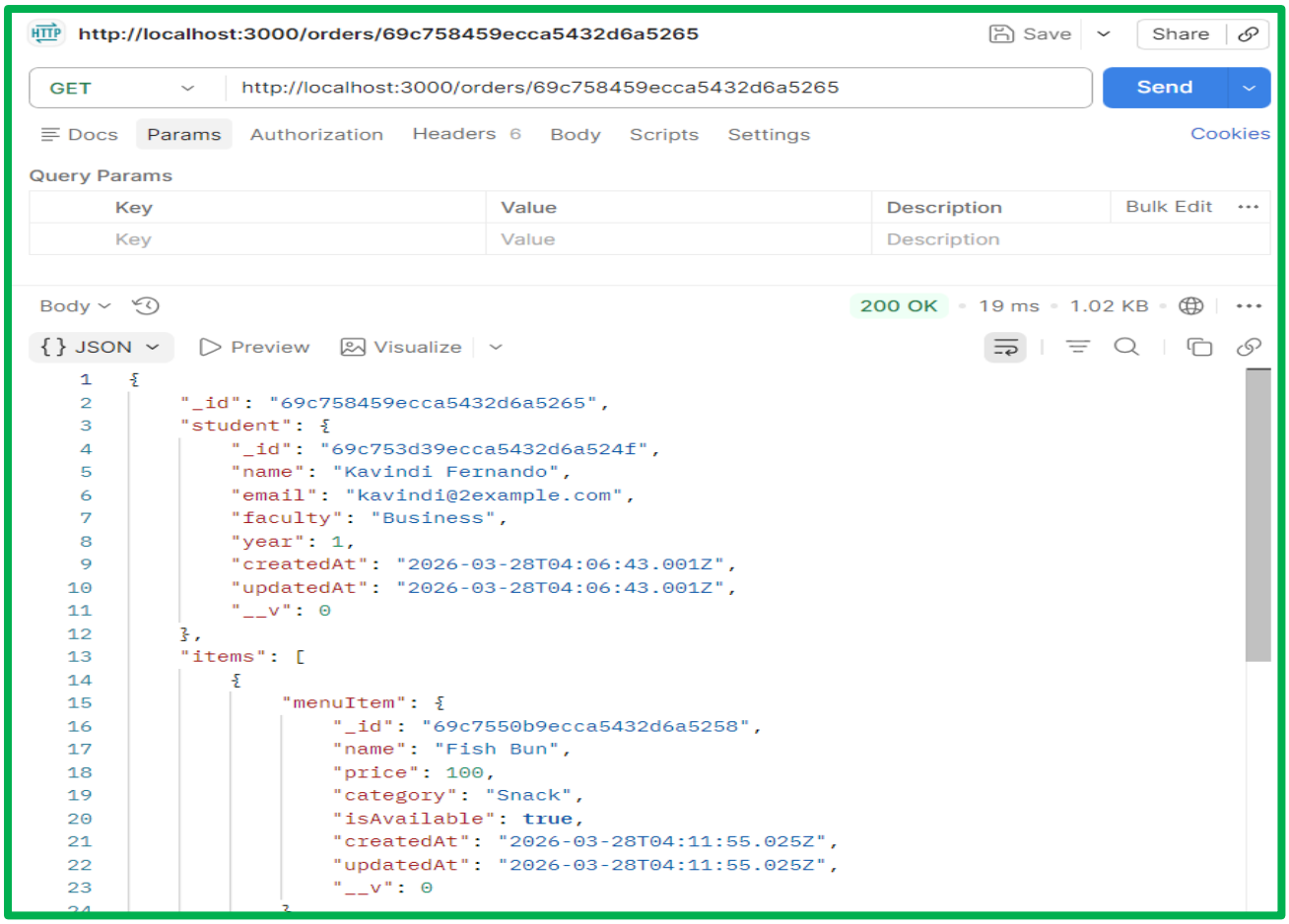


Place an order

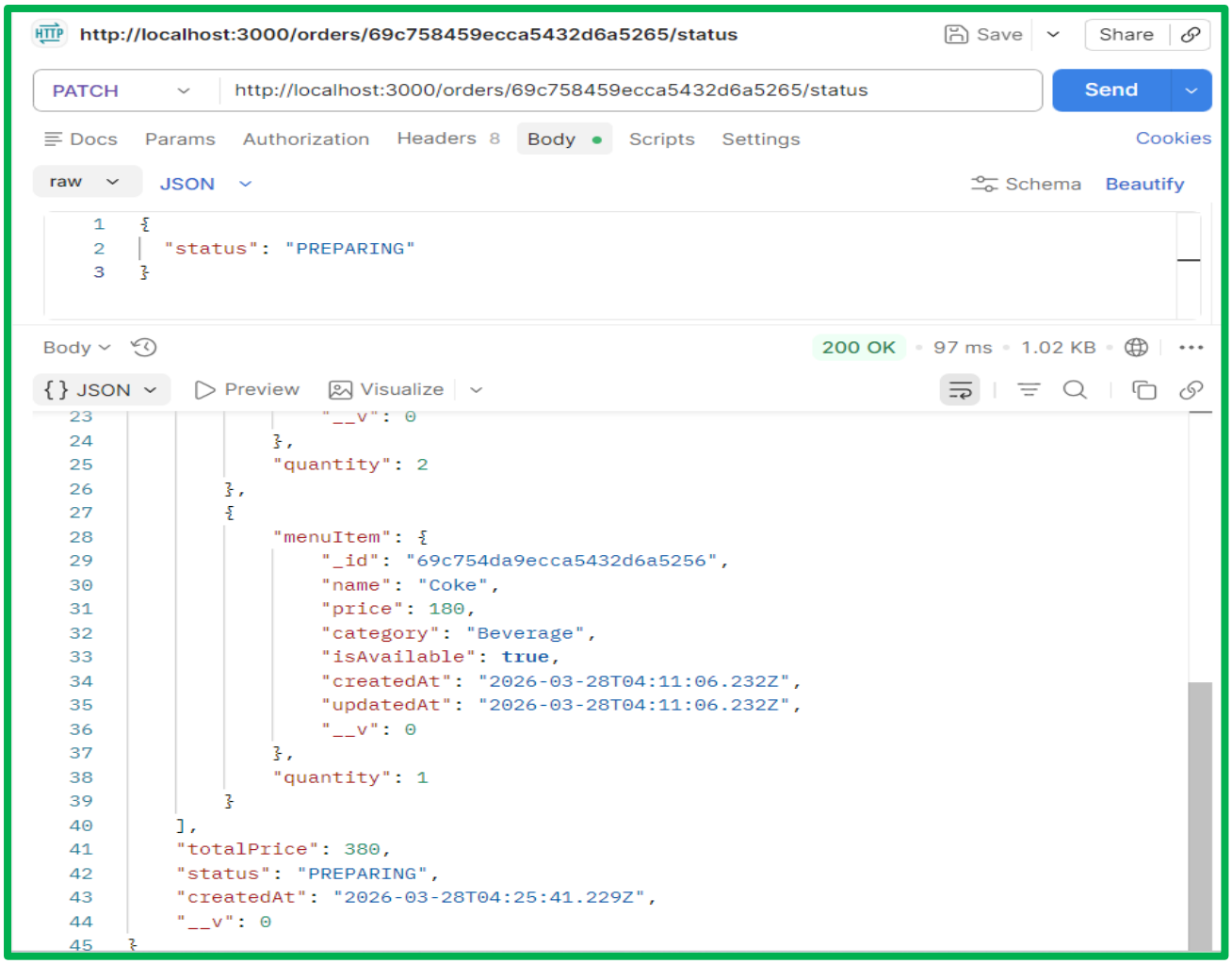




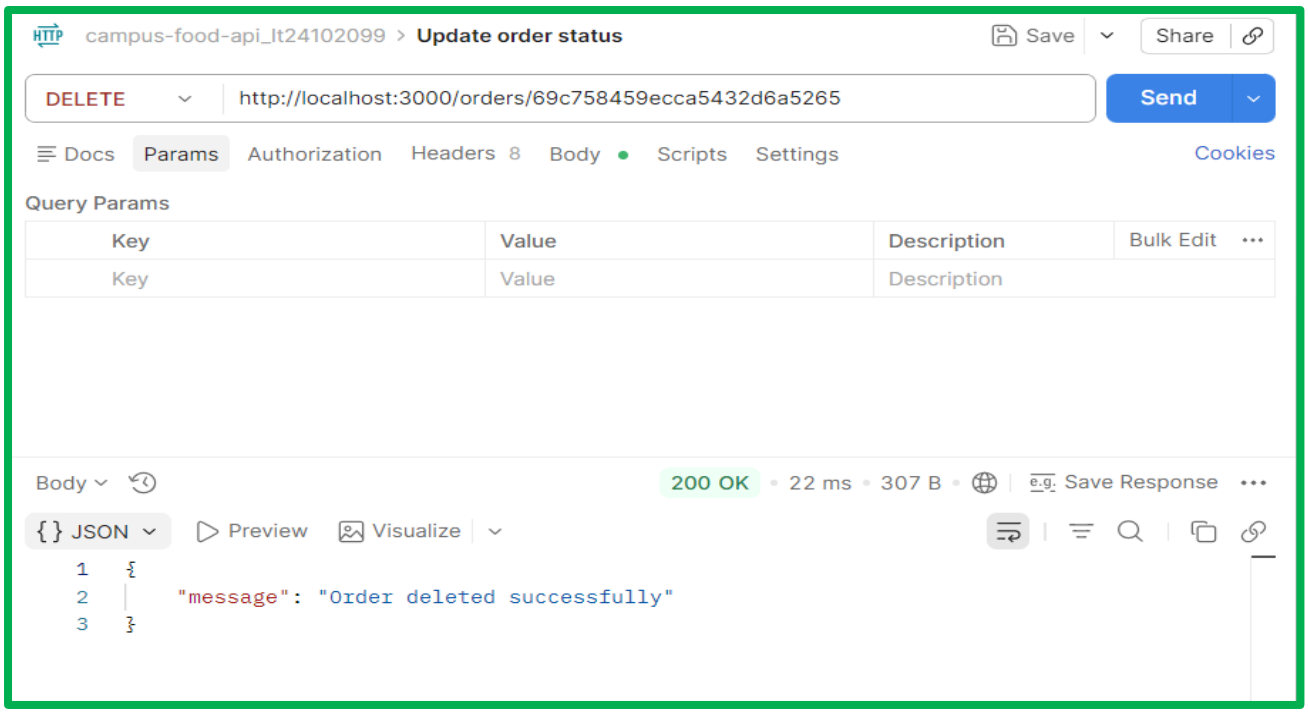
Get order by ID



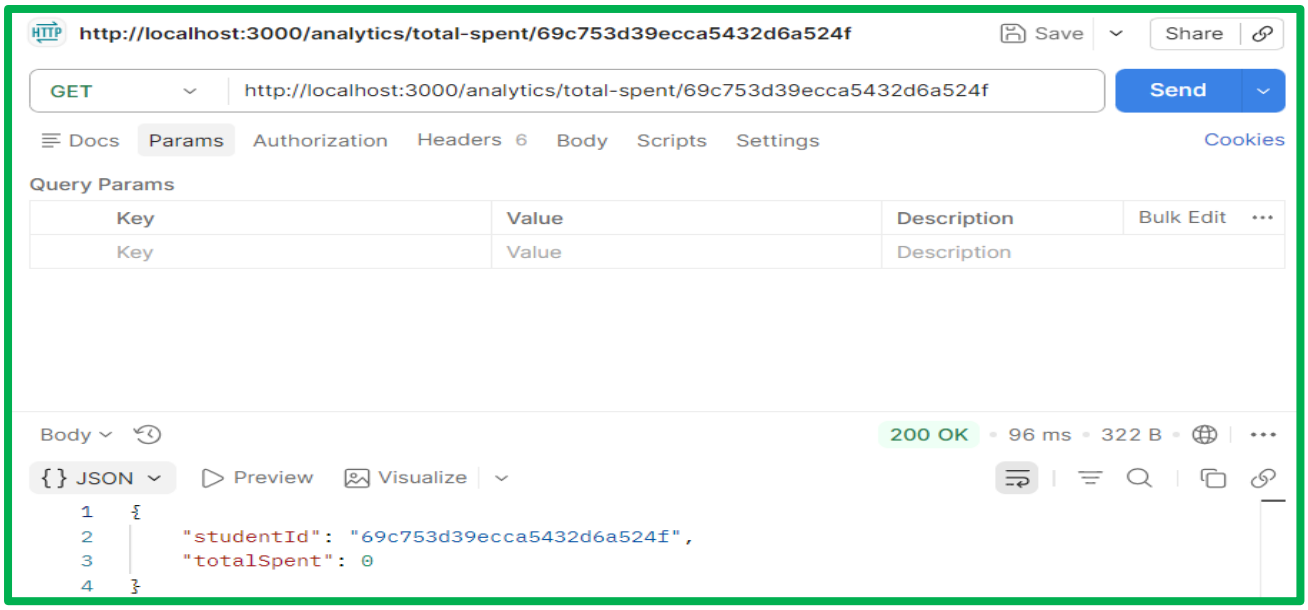
Update order status



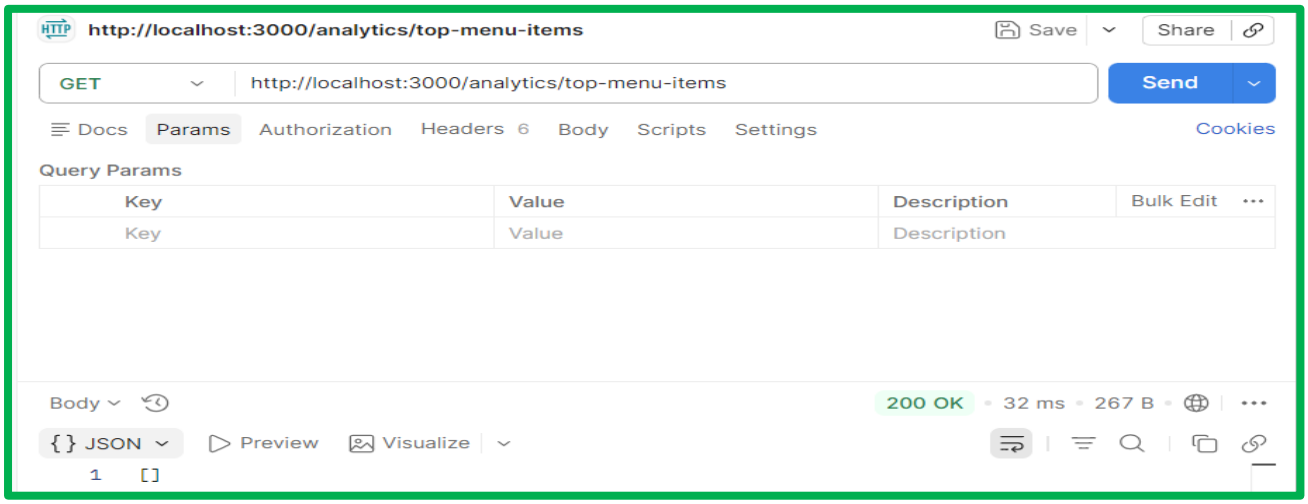
Delete order



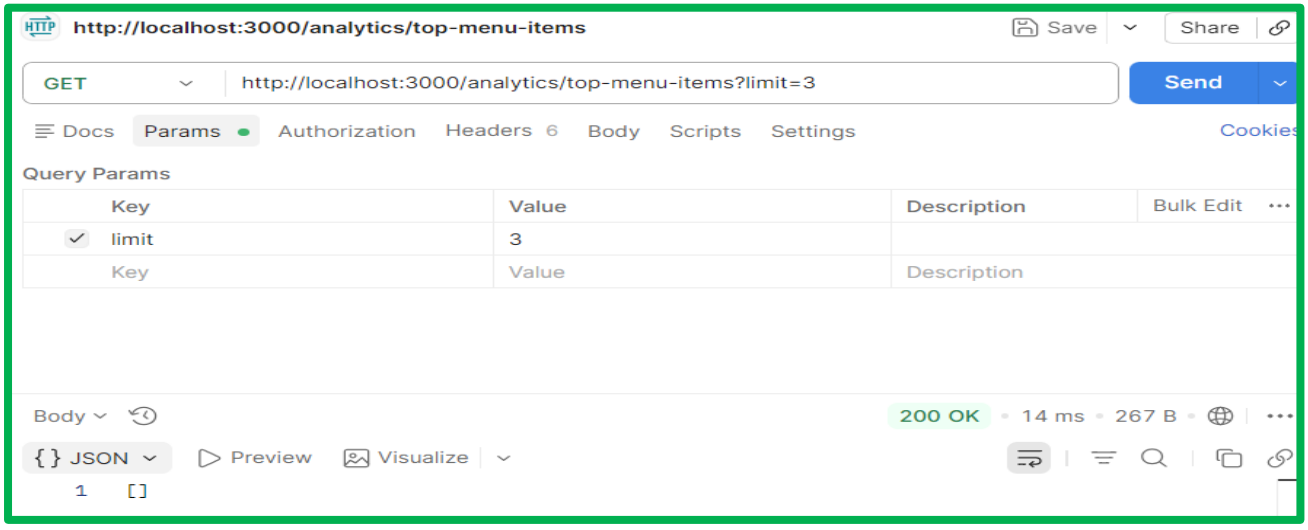
Analytics: total spent by a student



Analytics: top menu items



Analytics: top menu items with limit



Analytics: daily orders